MIAE 215 · Programming for Mechanical & Industrial Engineers

Official Midterm 1-Page Document Cheatsheet & Exam Survival Guide

Concordia University · Gina Cody School of Engineering · Department of MIAE

1. Data Types, Memory Bounds & Literals

Type	Bytes	Range / Precision	Key Exam Behavior / Concordia Traps
`bool`	**1**	`true` (1) or `false` (0)	Any non-zero integer is `true` (`-1` is `true` !). Only `0` is `false` .
`char`	**1**	-128 to 127 (`'0'` =48, `'A'` =65, `'a'` =97)	Single quotes (`'a'`). Promotion to int: `'a' + 1 == 98` .
`int`	**4**	$-2,147,483,648$ to $+2,147,483,647$	**Integer Division Truncates**: `7 / 2 == 3` , `1 / 3 == 0` . Overflow wraps negative!
`float`	**4**	$\approx 1.2 \times 10^{-38}$ to 3.4×10^{38} (7 digits)	Round-off error. Overflow = `+/-inf` , Underflow = `0.0` , Div by 0 = `inf` .
`double`	**8**	$\approx 2.3 \times 10^{-308}$ to 1.7×10^{308} (15-16 dig)	Default literal (`3.14`). Always use `eps` for comparison, NEVER `==` !
`sizeof()`	Op	Returns size in bytes	`sizeof(int) == 4` , `sizeof(double) == 8` , `sizeof(char) == 1` .

⚠️ **Floating Equality Trap**: Never write `if (a == b)` . Always use tolerance:

```
double eps = 1.0e-9;
if (abs(a - b) < eps) { /* roughly equal */ }
```

2. Operator Precedence & Evaluation Rules

1. **Highest**: `()` `[]` `x++` `x--`
   (Left-to-Right)
2. **Unary**: `++x` `--x` `+` `-` `!` `&` (address-of) `*` (dereference) `sizeof` `(cast)`
   (Right-to-Left)
3. **Multiplicative**: `*` `/` `%`

(Left-to-Right)

→ Modulo `%` requires integers: `7 % 3 == 1`

4. **Additive**: `+` `-`

   (Left-to-Right)

5. **Relational**: `<` `<=` `>` `>=`

   (Left-to-Right)

6. **Equality**: `==` `!=`

   (Left-to-Right)

7. **Logical AND**: `&&`

   (Left-to-Right, Short-circuit: if LHS false, RHS skipped)

8. **Logical OR**: `||`

   (Left-to-Right, Short-circuit: if LHS true, RHS skipped)

9. **Conditional (Ternary)**: `cond ? expr1 : expr2`

   (Right-to-Left)

10. **Assignment**: `=` `+=` `-=` `*=` `/=` `%=`

    (Right-to-Left)

> 🧠 **Prefix vs. Postfix**:
>
> - `y = x++;` ⟹ `y` gets current `x`, **then** `x` increments by 1.
> - `y = ++x;` ⟹ `x` increments by 1 **first**, then `y` gets new `x`.

---

3. Dr. Gordon's Table Tracing Protocol (Signature Question 2)

1. **Trace Columns**: Draw one column per variable (`i`, `j`, `sum`, `A[0]`, `A[1]`, etc.) + an **Output** column.
2. **Line-by-Line State**: Write a number **only when that variable updates**. Leave cells blank if unchanged.
3. **Cout Trigger**: The moment `cout` executes, append characters immediately into the Output column.
4. **Array Boundary / Garbage (`G`) Trap**: For `double A[3]`, valid indices are `0, 1, 2`. If a loop terminates at `i = 3` and prints `A[i]`, it accesses unallocated memory → **Output is Garbage (`G`)**!

---

4. 1D & 2D Arrays & Matrix Manipulation

```
// 1. Partial Initialization (remaining elements zeroed):
double B[3] = {0.0}; // All 3 elements are 0.0
double M[][3] = {1, 2, 3, 4, 5, 6}; // Column size MUST be specified!

// 2. Matrix Transpose: row becomes column
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        tA[i][j] = A[j][i];
    }
}

// 3. Array Reversal:
for (int i = 0; i < n; i++) {
    revA[i] = A[n - 1 - i];
}

// 4. Value Swap:
double temp = A[i]; A[i] = A[j]; A[j] = temp;
```

---

5. C-Strings (`char str[]`) & Space Removal

- **Null Terminator**: Strings MUST end with `\0`. `char s[6] = "apple";` has `s[5] == '\0'`.
- **String Length without** `<cstring>`: `int i = 0; while (s[i] != '\0') i++; int len = i;`
- **Space Removal Function (** `no_space` **)**:

```
void no_space(char str1[]) {
    int i, newsize = 0;
    for (i = 0; str1[i] != '\0'; i++) {
        if (str1[i] != ' ') {
            str1[newsize++] = str1[i];
        }
    }
    str1[newsize] = '\0'; // Crucial: Re-terminate with null character!
}
```

---

6. File Input / Output (`<fstream>`)

Step-by-Step Writing (`ofstream`)

```
#include <fstream>
ofstream fout("A.txt");
if (!fout) { cout << "\noutput file open error"; return 1; }
fout << scientific;
fout.precision(9); // 9 decimal places = 10 digits
for (int i = 0; i < N; i++) fout << A[i] << "\n";
fout.close();
```

Step-by-Step Reading (`ifstream`)

```
ifstream fin("A.txt");
if (!fin) { cout << "\ninput file open error"; return 1; }
for (int i = 0; i < N; i++) fin >> B[i];
fin.close();
```

---

7. Pointers & Dynamic Memory (`new` / `delete[]`)

- `&b` returns memory address of `b`. `*p` dereferences (accesses value at address `p`).
- Pointer arithmetic: `p + 1` moves by `sizeof(*p)` bytes (+4 for int, +8 for double).
- Array-pointer equivalence: `A[i] == *(A + i) == p[i]`.

```
// Dynamic 1D Array Allocation:
int N; cin >> N;
double *A = new double[N];
if (A == NULL) { cout << "\ndynamic allocation error"; return 1; }

for (int i = 0; i < N; i++) A[i] = sin(1.1 * i);

delete[] A; // Crucial: Always deallocate dynamic memory to avoid memory leaks!
```

---

8. Functions: Call-by-Value vs. Call-by-Reference

```
void swapVal(int a, int b) { int t = a; a = b; b = t; }   // Call-by-Value: copies only! Caller
unchanged.
void swapRef(int &a, int &b) { int t = a; a = b; b = t; } // Call-by-Ref: aliases caller variables!
void passPtr(int *a, int *b) { int t = *a; *a = *b; *b = t; } // Pointer-Ref: caller passes &x, &y
```

---

9. Flowgorithm / Flowchart Standard Symbols

- **Oval / Capsule**: Start / End / Return.
- **Parallelogram**: Input (`cin >> x`) / Output (`cout << y`).
- **Rectangle**: Process / Assignment (`sum = sum + i` , `int i = 0`).
- **Diamond**: Decision (`if (i < n)` , `while condition`) with True / False arrows.

---

10. Top Concordia Exam Pitfalls & Zero-Marks Traps

1. `if (x = 5)` instead of `if (x == 5)` $\rightarrow$ Assigns 5, evaluates to true, destroys variable!
2. `switch(val)` on a `double` $\rightarrow$ Compiler Error! Switch only accepts integral types (`int` , `char` , `bool`).
3. Forgetting `break;` in `switch` $\rightarrow$ Falls through all remaining cases!
4. `for (int i = 0; i <= n; i++)` on `A[n]` $\rightarrow$ Iteration `n` accesses out-of-bounds memory $\Longrightarrow$ Garbage (`G`)!
5. Forgetting `fout.close()` or `delete[] p` $\rightarrow$ File lock or memory leak penalty.
6. Initializing `maxVal = 0` $\rightarrow$ If all array elements are negative, `maxVal` returns incorrect 0! Initialize to `A[0]` .